

로봇주행가능성 정보 적용을 위한 코스트맵 프레임워크 경량화

Lightweight Costmap Framework for Applying Traversability Information

이민호·박윤수·김보메나·심인욱[†]

Minho Lee, Younsoo Park, Bomena Kim, Inwook Shim[†]

Abstract: In real-world robotic deployments, robust and high-speed traversability mapping is essential for reliable navigation. This work presents a lightweight costmap framework that converts per-pixel traversability predictions from RGB images into a robot-centric 2D gridmap. The proposed pipeline time-synchronizes depth, traversability, and odometry, reconstructs a body-frame point cloud, attaches per-point traversability values via camera reprojection, and applies voxel downsampling to reduce redundancy and sensor noise before projecting the result onto a 2D costmap compatible with the ROS 2 navigation stack. To reduce computation, we convert a PyTorch-based traversability network into an optimized TensorRT engine, enabling the framework to maintain real-time gridmap updates within the compute and memory budget of an embedded platform.

Keywords: Traversability, Costmap, Embedded system

1. 서론

실제 운용되는 자율주행 로봇은 단일 임베디드 시스템 상에서 내비게이션, 인지, 상호작용 등 복수의 기능을 동시에 수행해야 하므로, 연산 자원에 대한 제약을 받는다. 최근 로봇 주행에서는 고해상도 맵 표현과 복잡한 딥러닝 기반 인지가 함께 사용되면서, 이를 제한된 메모리 예산 안에서 처리할 수 있도록 경량화된 시스템 구성이 요구된다.

이 가운데, 주행가능성(traversability)을 얼마나 정밀하게 추정하는가는 특히 근거리 환경에서 매우 중요하다. 근거리에서의 부정확한 주행가능성 평가는 곧바로 장애물 접촉 및 충돌 위험으로 이어지기 때문이다. 이러한 필요에 대응하기 위해, 복잡한 지형과 장애물을 픽셀 단위로 분류하는 딥러닝 기반 주행가

능성 추론 기법이 제안되고 있으며, GPU 기반 elevation mapping[1]이나 OctoMap[2]과 같은 3차원 맵 표현 방식 역시 지형과 장애물을 정밀하게 모델링하기 위해 활용되어 왔다. 이들 접근은 표현력과 추론 정밀도 측면에서 분명한 장점을 가지지만, 경량화 관점에서 보면 실시간 주행 로봇의 근거리 환경을 표현할 때 고해상도 3차원 누적 지도와 대규모 신경망 추론은 임베디드 시스템이 감당하기 어려운 계산 및 메모리 비용을 요구한다는 근본적인 한계를 지닌다.

본 연구는 이러한 경량화 문제를 직접적으로 다루기 위해, 실제 임베디드 자율주행 시스템의 제약 조건을 전제로 한 경량 주행가능성 코스트맵 프레임워크를 제안한다. 3차원 누적 지도를 유지하는 대신, 로봇 중심(robot-centric) 2차원 격자를 기반으로 하는 코스트맵 표현을 채택하고, 딥러닝 기반 주행가능성 추론을 ONNX 및 TensorRT 기반으로 경량화함으로써, 임베디드 보드 환경에서도 실시간 근거리 주행가능성 정보를 활용할 수 있도록 설계한다. 제안하는 프레임워크는 누적 지도를 사용하지 않는 경량 구조를 통해 연산 부담을 최소화하면서도, ROS 2 내비게이션 스택과 직접 연동 가능한 형태의 주행가능성 코스트맵을 제공하는 것을 목표로 한다.

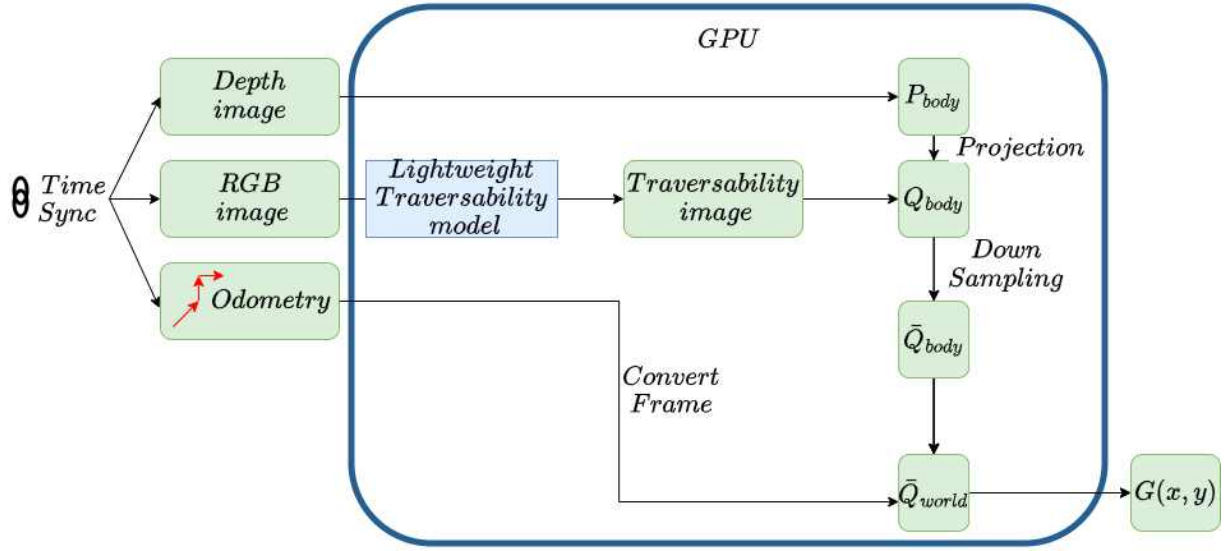
※ This research was supported by the National Research Foundation of Korea-(NRF) grant funded by the Korea government-(MSIT)-(RS-2024-00342176 and RS-2025-02217000)

Minho Lee, Younsoo Park, Bomena Kim, Inwook Shim, RCVlab, Inha University, Incheon, Korea

(mhlee00@inha.edu, yspark98@inha.edu, brnkmkim@inha.edu, iwshim@inha.ac.kr)

[†] Inwook Shim, Corresponding author: Smart Mobility Engineering,

Inha University, Incheon, Korea, RCVLab. (iwshim@inha.ac.kr)



[Fig. 1] Mapping Framework

2. 본 론

2.1 경량화 기법

본 연구에서 사용한 주행가능성 추론 모듈은 2차원 이미지 입력을 기반으로 픽셀 단위 주행가능성을 예측하는 모델이다. 연산 부하를 줄이기 위해, PyTorch 기반 주행가능성 추론 모델을 ONNX로 변환한 뒤 TensorRT 엔진으로 사전 컴파일하여 추론 과정에서 불필요한 연산을 제거하였다. 최종적으로 주행가능성 추론은 고정된 실행 경로에서 일정한 지연 시간으로 수행되며, 이 결과는 로봇 중심 맵과 결합되어 전체 코스트맵 생성의 실시간성을 유지하는 데에 사용된다.

2.2 매핑 프레임워크

$$P_{body} = \{p_i\}_{i=1}^N \quad \text{where } p_i = (x_i, y_i, z_i) \quad (1)$$

$$Q_{body} = \{p_i, t_i\}_{i=1}^N, t_i \in [0, 1] \quad (2)$$

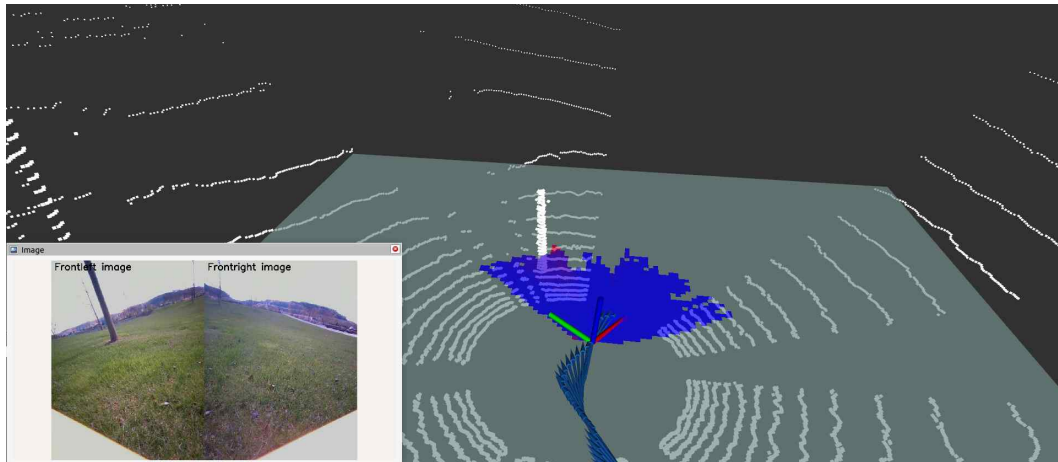
$$\bar{Q}_{body} = \{\bar{p}_k, \bar{t}_k\}_{k=1}^M \quad (3)$$

$$\bar{Q}_{world} = T_{body} \cdot \bar{Q}_{body} \quad (4)$$

$$G(x, y) = (1 - t_{(x, y)}) \times 100, G(x, y) \in [0, 100] \quad (5)$$

Fig. 1은 본 연구에서 제안하는 코스트맵 프레임워크의 구조를 나타낸다. 입력으로는 깊이 이미지, 컬러 이미지, 그리고 오도메트리 정보가 사용되며, 최종 출력은 2차원 격자 코스트맵이다. 세 입력은 시간 동기화된 후, 깊이 이미지는 가우시안 필터링을 통해 노이즈가 제거되고 이후 필터링된 깊이로부터 바디

프레임 기준 3차원 포인트클라우드를 생성한다. 이때 하나의 점은 $p_i = (x_i, y_i, z_i)$ 형태로 구성되며, 포인트클라우드는 N개의 포인트 집합으로, 식 (1)의 P_{body} 로 표현할 수 있다. 컬러 이미지는 경량화된 주행가능성 추론 모듈을 통해 픽셀 단위 주행가능성으로 변환되어 주행가능성 이미지로 생성된다. 주행가능성 이미지로부터 픽셀 단위의 주행가능성 $t_i \in [0, 1]$ 을 P_{body} 와 대응시켜 포인트클라우드의 각 점 p_i 에 부여하면, 식 (2)의 Q_{body} 와 같이 주행가능성 속성이 결합된 포인트클라우드를 얻을 수 있다. 여기서 $t_i = 1$ 은 완전 주행 가능, $t_i = 0$ 은 주행 불가능을 의미한다. 이후 공간 중복과 노이즈를 제거하기 위해 복셀 다운샘플링을 수행하며, 동일 복셀 내의 점들은 평균 위치와 평균 주행가능성으로 대표되어 다운샘플링된 k개의 값으로 식 (3)의 $\bar{Q}_{body}$ 로 축약된다. 오도메트리 정보에서 획득한 포즈로부터 변환 행렬을 구하고, 이를 통해 식 (4)와 같이 $\bar{Q}_{body}$ 를 월드 프레임으로 변환하여 $\bar{Q}_{world}$ 로 나타낼 수 있다. 변환된 $\bar{Q}_{world}$ 는 평면 격자에 투영되며, 이후 주행 가능도를 코스트로 변환하여 각 셀에서 가장 낮은 값 하나만을 남긴 후 식 (5)의 코스트맵 $G(x, y)$ 로 나타낸다. $G(x, y)$ 는 셀 단위 코스트를 의미하며, 0은 완전 주행 가능, 100은 주행 불가능, 관측되지 않은 셀은 -1로 초기화된다. $G(x, y)$ 는 로봇 중심의 코스트맵으로 퍼블리시되며, 해당 코스트맵을 rviz로 시각화 결과는 Fig. 2처럼 나타낼 수 있다. 포인트클라우드 처리는 Pytorch 기반 GPU 연산으로 진행된다.



[Fig. 2] Costmap example

2.3 실험

2.3.1 실험 환경

실험은 Jetson AGX Orin 64GB에서 수행하였다. 센서는 Intel RealSense D435 카메라 2대를 사용하였고, 15 Hz의 RGB-D 이미지를 수신하였다. 오도메트리 데이터는 50Hz로 수집되었으며, 모든 모듈은 타임스탬프 기반으로 동기화하여 파이프라인에 투입하였다. 플랫폼은 Boston Dynamics Spot을 사용하였으며, 아웃도어 환경의 자체 구축 데이터셋에서 평가를 수행하였다. 격자 해상도는 0.10m, 맵 크기 150×150 셀, 복셀 크기 0.05m로 설정하였다.

2.3.2 실험 결과

단일 프레임워크의 실시간 연산시간(ms) 과 GPU 메모리 점유율(%) 및 메모리 사용량(GB)로 결과를 요약한다. 제안 파이프라인은 실험 환경에서 약 150 ms의 연산시간을 보였다. 이 중 이미지 기반 주행가능성 추론에 소요되는 시간은 약 100 ms이다. 이는 동일 조건에서 기존 PyTorch 기반 모델의 연산 시간이 약 570 ms였던 것과 비교할 때, 추론 모듈이 상당히 효과적으로 경량화되었음을 보여준다.

GPU 메모리 점유율은 평균 48.10%, 표준편차 44.05%로 관측되었으며, 측정값중 점유율 80% 이상을 차지한 값이 약 40%, 점유율 20% 미만을 차지한 값이 약 40%로 측정되었다. GPU 메모리 사용량은 약 16GB로 측정되었다. 두 지표를 종합하면, 본 프레임워크는 야외 주행 시 요구되는 15 Hz의 RGBD 카메라와 50Hz의 오도메트리 입력을 실시간으로 소화하면서, 임베디드 환경에서 장시간 동작에 충분한 메모리 안정성을 보여준다.

[Table 1] Computation performance test

		평균	표준편차
GPU	mem(%)	48.10	44.05
GPU	mem(GB)	16	0.05

3. 결 론

본 연구는 주행가능성 정보를 실시간 코스트맵으로 변환하는 경량 프레임워크를 제안하였다. 깊이-주행가능성-오도메트리를 시간 동기화한 뒤 2D 코스트맵으로 안정적으로 생성한다. Jetson AGX Orin 64GB에서 15 Hz의 RGBD 이미지와 50 Hz의 오도메트리 입력을 실시간 처리하며 150 ms로 격자 지도를 생성하였고, GPU 메모리 사용량 평균 16GB, GPU 메모리 점유율 평균 48.10%로 운용 안정성을 확인했다.

References

- [1] Takahiro Miki et al., "Elevation Mapping for Locomotion and Navigation using GPU", In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Kyoto, Japan, pp. 2273 -2280, 2022.
- [2] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Auton. Robots*, Vol. 34, No. 3, pp. 189-206, 2013